

NIST – Cloud Computing Definition

Cloud Computing ist ein Modell für den bedarfsgerechten Netzwerkzugriff auf einen gemeinsam genutzten Pool konfigurierbarer IT-Ressourcen (Netzwerke, Server, Speicher, Anwendungen, Dienste), die schnell bereitgestellt und mit minimalem Verwaltungsaufwand freigegeben werden können.

5 Merkmale einer Cloud (NIST)

# Merkmal	Kurzbeschreibung
1 On-Demand Self-Service	Ressourcen selbst buchen, ohne menschlichen Eingriff beim Anbieter
2 Broad Network Access	Zugriff über Netzwerk mit Standardgeräten (PC, Handy, etc.)
3 Resource Pooling	Ressourcen werden mehreren Kunden geteilt (Multi-Tenant)
4 Rapid Elasticity	Ressourcen schnell hoch- und runterskalieren
5 Measured Service	Nutzung wird gemessen und abgerechnet (Pay-per-Use)

Cloud Deployment Modelle

Modell	Beschreibung
Public Cloud	Infrastruktur vom Anbieter (AWS, Azure, GCP), öffentlich zugänglich
Private Cloud	Exklusiv für eine Organisation, on-premise oder extern gehostet
Hybrid Cloud	Kombination aus Public und Private Cloud
Community Cloud	Geteilt von mehreren Organisationen mit gleichen Anforderungen (z.B. Behörden)
Multi-Cloud	Nutzung mehrerer Cloud-Anbieter gleichzeitig

Cloud Service Modelle

Modell	Vollname	Du verwaltest	Anbieter verwaltet	Beispiel
IaaS	Infrastructure as a Service	OS, Apps, Daten	Hardware, Netzwerk, Virtualisierung	AWS EC2, Azure VM
PaaS	Platform as a Service	Nur App & Daten	Alles darunter (OS, Runtime, etc.)	Heroku, Google App Engine
FaaS	Function as a Service	Nur den Code / die Funktion	Alles (Infrastruktur, Scaling, Runtime)	AWS Lambda, Azure Functions
SaaS	Software as a Service	Nichts	Alles	Microsoft 365, Gmail

Grundlagen Cloud Computing

Bekannte Cloud-Dienstleistungen

- Compute (virtuelle Maschinen, Serverless Functions)
- Storage (Objektspeicher, Blockspeicher)
- Datenbanken (managed SQL/NoSQL)
- Netzwerk (CDN, Load Balancer, VPN)
- KI/ML-Dienste
- Container-Dienste (Kubernetes as a Service)
- Monitoring & Logging

Cloud-Anbieter (grobe Übersicht)

Anbieter	Kurzprofil
AWS (Amazon)	Grösster Anbieter, riesiges Angebot, sehr verbreitet, Marktführer
Azure (Microsoft)	Stark in Unternehmensumgebungen, gute Integration mit Windows/Office
GCP (Google)	Stark in KI/ML und Big Data, Kubernetes stammt von Google
IBM Cloud	Fokus auf Enterprise, KI (Watson), Hybrid Cloud
Oracle Cloud	Stark bei Datenbanken und Enterprise-Software
Exoscale / Cloudscale	Schweizer Anbieter, DSGVO-konform, kleinere Unternehmen
Hetzner	Günstiger europäischer Anbieter, vor allem VPS/Server

Monitoring vs. Logging

	Monitoring	Logging
Was?	Echtzeit-Überwachung von Metriken (CPU, RAM, Verfügbarkeit)	Aufzeichnung von Ereignissen/Aktionen über Zeit
Ziel	Probleme sofort erkennen & Alarm auslösen	Fehleranalyse, Nachvollziehbarkeit, Audit
Beispiel	CPU > 90% → Alert	„User X hat sich um 14:32 eingeloggt“
Tools	Prometheus, Grafana, Datadog	ELK-Stack, Loki, Splunk

Vorteile der Cloud

- **Kosteneffizienz** – kein eigenes Rechenzentrum, Pay-per-Use
- **Skalierbarkeit** – schnell hoch- und runterskalieren
- **Verfügbarkeit** – hohe Uptime-Garantien (SLA)
- **Globalität** – Rechenzentren weltweit
- **Kein Hardware-Unterhalt** – Updates und Wartung durch Anbieter
- **Schnelle Bereitstellung** – Infrastruktur in Minuten verfügbar
- **Innovation** – Zugang zu KI, ML, Big Data ohne Eigenentwicklung

Nachteile der Cloud

- **Datenschutz/Compliance** – Daten liegen beim Anbieter (DSGVO-Fragen)
- **Abhängigkeit (Vendor Lock-in)** – Wechsel zu anderem Anbieter ist aufwendig
- **Laufende Kosten** – Langfristig teurer als Eigeninfrastruktur möglich
- **Internetabhängigkeit** – Kein Netz = kein Zugriff
- **Kontrollverlust** – Weniger Kontrolle über physische Infrastruktur
- **Sicherheitsrisiken** – Shared Infrastructure, Angriffsfläche durch Internet

Shared Responsibility Model

Idee: Verantwortung wird zwischen Cloud-Anbieter und Kunde aufgeteilt – je nach Service-Modell anders.

Verantwortung	IaaS	PaaS	SaaS
Physische Infrastruktur	Anbieter	Anbieter	Anbieter
Betriebssystem	Kunde	Anbieter	Anbieter
Applikation	Kunde	Kunde	Anbieter
Daten	Kunde	Kunde	Kunde
Identitäten/Zugriffsrechte	Kunde	Kunde	Kunde

Merksatz: Der Anbieter sichert die Cloud selbst – der Kunde sichert, was er darin ablegt.

Grundlagen Container-Technologie

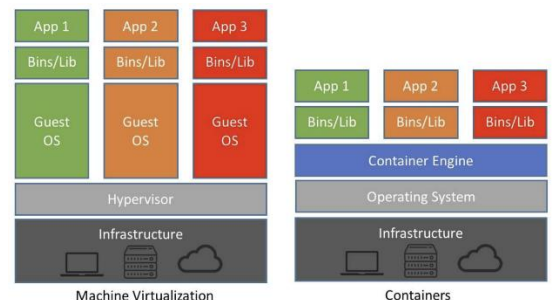
Was ist Container-Technologie?

Container-Virtualisierung ist eine Methode, Anwendungen mit allen ihren Abhängigkeiten (Libraries, Config, Runtime) in einem isolierten Paket – dem **Container** – auszuführen. Container teilen sich den **Kernel des Host-Betriebssystems**, laufen aber voneinander isoliert.

ABBILDUNG: Hypervisor simuliert für jede VM eigene Hardware, Container Engine hat gleiche Rolle ist aber leichtgewichtiger

Container vs. VM – Vor- und Nachteile

	VM	Container
Grösse	Gross (GB, ganzes OS)	Klein (MB, nur App + Libs)
Startzeit	Minuten	Sekunden
Isolation	Stark (eigener Kernel)	Weniger stark (shared Kernel)
Portabilität	Eingeschränkt	Sehr hoch
Ressourcen	Viel overhead	Effizient
Einsatz	Legacy-Apps, unterschiedliche OS Microservices, skalierbare Apps	



VM vs. Container – Technologievergleich

Kriterium	VM	Container
Bereitstellung	Langsamer (Image + OS booten)	Sehr schnell (Sekunden)
Speicherplatz	Gross (ganzes OS enthalten)	Klein (nur App & Dependencies)
Portabilität	Eingeschränkt (Hypervisor-abhängig)	Sehr hoch (läuft überall mit Docker)
Effizienz	Niedriger (OS overhead)	Hoch (shared Kernel)
Betriebssystem	Eigener Kernel pro VM	Shared Kernel des Hosts

Bekannte Produkte

Bereich	Produkte
VMs	VMware, VirtualBox, Hyper-V, KVM, AWS EC2, Azure VM
Container	Docker, Podman, containerd
Orchestrierung	Kubernetes, Docker Swarm, OpenShift

Können Container VMs immer ersetzen?

Nein. Container sind nicht für alle Anwendungsfälle geeignet:

- Apps die ein **bestimmtes OS** brauchen (z.B. Windows-App auf Linux-Host) → VM
- Apps mit **starken Isolation-Anforderungen** (Security) → VM bevorzugt
- **Legacy-Anwendungen** die tief ins OS eingreifen → VM
- Für moderne, zustandslose Microservices → Container ideal

Container / Image / Registry

Begriff	Erklärung
Image	Unveränderliche Vorlage/Bauplan für einen Container (wie eine ISO-Datei)
Container	Laufende Instanz eines Images – der tatsächlich ausgeführte Prozess
Registry	Speicherort für Images (z.B. Docker Hub, GitHub Container Registry, AWS ECR)

Analogie: Image = Kochrezept / Container = das fertige Gericht / Registry = Rezeptbuch

Warum sind laufende Container immutable (unveränderbar)?

- Änderungen im Container gehen beim Neustart **verloren** (kein persistenter Zustand)
- **Reproduzierbarkeit** – jeder Container startet immer gleich aus dem Image
- **Sicherheit** – kein unerwünschter Zustand durch Laufzeitänderungen
- **Best Practice:** Änderungen → neues Image bauen, nicht laufenden Container patchen
- Persistente Daten werden in **Volumes** ausgelagert

Self-Managed vs. Fully Managed

	Self-Managed	Fully Managed
Kontrolle	Voll (du konfigurierst alles)	Eingeschränkt (Anbieter entscheidet)
Aufwand	Hoch (Updates, Sicherheit, Scaling selbst)	Gering (Anbieter übernimmt)
Flexibilität	Sehr hoch	Weniger
Kosten	Tiefer in Ressourcen, höher in Personalaufwand	Meist teurer, aber einfacher
Beispiel	Kubernetes selbst installiert	AWS EKS, Google GKE, Azure AKS
Nachteil	App muss zustandslos sein	Irgendwann physisches Limit

Grundlagen Container-Orchestrierung

Warum braucht man Container-Orchestrierung?

Bei vielen Containern entstehen Probleme die man manuell nicht mehr lösen kann:

- Wo soll welcher Container deployed werden?
- Was passiert wenn ein Container abstürzt?
- Wie wird Last verteilt (Load Balancing)?
- Wie werden Updates eingespielt ohne Downtime?
- Wie viele Instanzen brauche ich gerade?

Orchestrierung löst das alles automatisch: Deployment, Scaling, Selbstheilung, Load Balancing – vollständig automatisiert.

Wie funktioniert Container-Orchestrierung?

Schritt für Schritt:

1. Du beschreibst den Soll-Zustand in einer YAML- oder JSON-Datei (z.B. „3 Instanzen von App X, Port 8080, Image von Registry XY“)
2. Das Orchestrierungs-Tool liest diese Konfiguration und plant das Deployment automatisch im Cluster
3. Es findet den richtigen Host, berücksichtigt Ressourcen und Einschränkungen
4. Es verwaltet dann den Lifecycle der Container gemäss den Spezifikationen
5. Soll- und Ist-Zustand werden laufend verglichen → Abweichungen werden automatisch korrigiert (z.B. Container neu starten)

Orchestrierung funktioniert überall: lokal, Private Cloud, Public Cloud.

Bekannte Orchestrierungs-Technologien

Technologie	Beschreibung
Kubernetes (K8s)	Industriestandard, sehr mächtig, komplex, horizontal skalieren ist seine Kernstärke
Docker Compose	Einfacher Einstieg, mehrere Container lokal zusammen definieren und starten
Docker Swarm	Direkt in Docker integriert, einfacher als K8s, weniger verbreitet
OpenShift	Kubernetes-basiert von Red Hat, Enterprise-fokussiert, vereinfacht Deployment & Management
Minikube	Leichtgewichtige lokale Kubernetes-Implementierung zum Lernen/Testen (1-Node-Cluster auf VM)
AWS ECS / EKS	Managed Container-Orchestrierung von Amazon

Kubernetes – wichtige Grundbegriffe

Begriff	Erklärung
Pod	Kleinste deploybare Einheit in K8s – enthält einen oder mehrere Container
Deployment	K8s-Objekt das die gewünschte Anzahl Pod-Replicas verwaltet und automatisch sicherstellt, dass diese laufen. Ermöglicht deklarative Updates und Rollbacks.
Service	Abstraktion die einen stabilen Netzwerk-Endpunkt für eine Gruppe von Pods bereitstellt – fungiert als interner Load Balancer und ermöglicht Service Discovery
Cluster	Gruppe von Nodes (Servern) auf denen K8s läuft
Node	Einzelner Server im Cluster

Begriff	Erklärung
Replica	Kopie eines Pods – mehrere Replicas = Hochverfügbarkeit

Kubernetes YAML-File – Aufbau

Kubernetes wird über YAML-Dateien konfiguriert. Die wichtigsten Felder:

Feld	Bedeutung
apiVersion	Version der Kubernetes-API die verwendet wird
kind	Typ der Ressource (z.B. Deployment, Service, Route)
metadata	Name, Namespace, Labels, Annotationen
spec	Gewünschter Zustand – z.B. Anzahl Replicas, Container-Image, Ports

Vertikale vs. Horizontale Skalierung

	Vertikal (Scale Up)	Horizontal (Scale Out)
Methode	Mehr CPU/RAM pro Pod	Mehr Pod-Instanzen starten
Einsatz	Eher für zustandsbehaftete Apps (Anfragen müssen zum selben Pod)	Eher für zustandslose Apps (jeder Pod ist gleich)
Cloud-geeignet	Begrenzt (Hardware-Limit)	Ja, ideal – K8s wurde dafür gebaut
Kubernetes	Möglich, aber nicht die Stärke	Kernstärke von K8s – skaliert innerhalb von Sekunden
Tool	–	HPA (Horizontal Pod Autoscaler) – min/max Pods + CPU/RAM-Schwellenwert definierbar

Deployment-Strategien

Strategie	Beschreibung	Downtime?	Vorteile	Nachteile
Basic (Recreate)	Alle Nodes gleichzeitig auf neue Version updaten	Ja	Einfach und schnell	Downtime, langsames Rollback, riskant
Rolling Update	Nodes schrittweise einzeln oder in Stapeln updaten	Nein	Einfach, geringeres Risiko als Basic, kein Unterbruch	Kurzzeitig zwei Versionen parallel, DB muss alt+neu unterstützen
Blue/Green	Zwei identische Umgebungen (Blue=Staging, Green=Production), Traffic per Switch umleiten	Nein	Sofortiges Rollback, sauber getrennt, kein Unterbruch	Doppelte Infrastrukturkosten, komplex bei Microservice-Abhängigkeiten
Canary	Neue Version zuerst nur für kleinen % der Nodes/User, schrittweise ausrollen	Nein	Geringstes Risiko, echtes Testen in	Komplex zu scripten, braucht gutes Monitoring (APM, Logs), DB-Kompatibilität beachten

Strategie	Beschreibung	Downtime? Vorteile	Nachteile
		Produktion, günstiger als Blue/Green	
12-Factor App			
Methodik für moderne, cloud-native Anwendungen. Ziel: skalierbar, wartbar, portabel.			
# Faktor	Was bedeutet das?	Wieso wichtig?	
1 Codebase	Ein Repo, viele Deployments	Kein Chaos mit mehreren Codebases	
2 Dependencies	Abhängigkeiten explizit deklarieren (z.B. package.json)	Keine „funktioniert nur auf meinem PC“-Probleme	
3 Config	Config in Umgebungsvariablen, nicht im Code	Sicherheit, verschiedene Umgebungen einfach	
4 Backing Services	Datenbanken, Queues etc. als austauschbare Ressourcen behandeln	Einfacher Wechsel (z.B. lokale DB → Cloud DB)	
5 Build, Release, Run	Klar getrennte Phasen: Build → Release → Run	Reproduzierbare, rollback-fähige Deployments	
6 Processes	App läuft als zustandslose Prozesse	Einfaches Skalieren, kein gemeinsamer Speicher	
7 Port Binding	App stellt Dienste selbst per Port bereit (kein ext. Webserver nötig)	Portabilität, unabhängig von Infrastruktur	
8 Concurrency	Skalierung durch mehr Prozesse (horizontal)	Cloud-nativ skalierbar	
9 Disposability	Schnelles Starten und sauberes Beenden	Robustheit, schnelles Deployment/Scaling	
10 Dev/Prod Parity	Entwicklung, Staging, Produktion möglichst gleich	Weniger „bei mir läuft's“-Bugs	
11 Logs	Logs als Ereignisstrom (stdout), nicht in Dateien	Zentrale Log-Verwaltung in der Cloud möglich	
12 Admin Processes	Einmalige Aufgaben (DB-Migration) als eigene Prozesse	Sauber, nachvollziehbar, nicht im App-Code versteckt	

Zusammenfassung: Die 12-Factor-App-Methodik sorgt dafür, dass Applikationen cloud-ready sind – sie lassen sich einfach skalieren, in verschiedenen Umgebungen betreiben, sind wartbar und robust gegen Ausfälle. Besonders wichtig für Container- und Kubernetes-Deployments.

Dockerfile NGINX static html site

nginx als Basis-Image – perfekt für statische HTML-Seiten

FROM nginx:alpine

Arbeitsverzeichnis setzen

WORKDIR /usr/share/nginx/html

index.html in den nginx-Webroot kopieren

COPY index.html /usr/share/nginx/html

Port 80 freigeben (Standard HTTP-Port von nginx)

EXPOSE 80

nginx startet automatisch mit dem nginx-Image, kein CMD nötig

Wieso kein CMD? Das nginx-Image hat bereits einen eingebauten Startbefehl. Man muss nur die Dateien an den richtigen Ort kopieren: /usr/share/nginx/html – das ist der Webroot von nginx.

Compose.yaml für Full Stack site

version: "3.9"

services:

frontend:

build:

context: ./frontend # Dockerfile liegt im /frontend Ordner

container_name: frontend

ports:

- "8080:80" # Website erreichbar über localhost:8080

volumes:

- ./frontend:/usr/share/nginx/html

Änderungen an HTML/CSS sofort sichtbar

depends_on:

- backend # Wartet bis Backend gestartet wurde

networks:

- frontend-netz # Kann mit Backend kommunizieren

restart: always # Container startet automatisch neu

backend:

build:

context: ./backend # Dockerfile liegt im /backend Ordner

container_name: backend

ports:

- "3000:3000" # API erreichbar über localhost:3000

environment:

- NODE_ENV=production

- DB_HOST=datenbank # Containername der DB als Hostname

- DB_PORT=5432

- DB_USER=admin

- DB_PASSWORD=geheim

- DB_NAME=meinedb

depends_on:

- datenbank # Wartet bis Datenbank gestartet wurde

networks:


```
- frontend-netz      # Kommunikation mit Frontend
- backend-netz       # Kommunikation mit Datenbank
restart: always
```

datenbank:

```
image: postgres:15      # Offizielles PostgreSQL Image
```

```
container_name: datenbank
```

environment:

```
- POSTGRES_USER=admin
- POSTGRES_PASSWORD=geheim
- POSTGRES_DB=meinedb
```

volumes:

```
- db-daten:/var/lib/postgresql/data
      # Daten bleiben persistent gespeichert
```

networks:

```
- backend-netz      # Nur Backend darf darauf zugreifen
```

```
restart: always
```

volumes:

```
db-daten:           # Speichert PostgreSQL Daten dauerhaft
```

networks:

frontend-netz:

```
driver: bridge      # Frontend ↔ Backend Kommunikation
```

backend-netz:

```
driver: bridge      # Backend ↔ Datenbank Kommunikation
      # DB ist NICHT direkt öffentlich erreichbar
```